数 值 分 析

数值逼近及常微分方程数值解法

邓 卫 兵　吴　　婷

编 写

徐 勤 武　顾 国 勇

School of Math, NJU

二 零 二 四 年 八 月

目　　录

第一章　数值计算中的误差分析初步 **1**
1.1　什么是数值方法 1
1.2　误差的基本概念 3
1.2.1　误差来源 3
1.2.2　绝对误差和相对误差 4
1.2.3　舍入误差与有效数字 5
1.3　机器误差 7
1.3.1　规格化的科学记数法 7
1.3.2　计算机中数的表示 7
1.3.3　机器舍入 10
1.3.4　浮点运算 12
1.3.5　数值运算的简单原则 14
1.4　算法的稳定性分析 15
1.4.1　算法的数值稳定性 15
1.4.2　病态问题与条件数 16
习题 一 17

附录 A　非线性方程的迭代解法 **20**

参考文献 **24**

School of Math, NJU

School of Math, NJU

第一章 数值计算中的误差分析初步

1.1 什么是数值方法

现实世界中的很多问题若需定量分析, 都要建立其对应的数学模型, 如非线性方程、线性与非线性代数方程组、常微分方程或偏微分方程等等, 从而将实际问题转化为数学问题的求解. 对于一些简单的数学问题, 可以用解析方法精确求解. 但是, 从自然科学和工程技术等领域中产生的许多数学问题, 往往得不到它们的解析解或者精确值, 从而需要设计寻求问题解的近似值的方法. 通常把这种研究并解决数学问题的近似解的方法称为**数值方法**. 数值方法关心的主题通常被称为数值分析. 不同的学科, 不同的工程应用会提出不同的数学问题, 建立的数学模型也会有所差别. 数值分析着重研究的是各个领域(特别是科学研和工程)中提出的各种数学模型的数值方法及有关理论. 由于计算机本身的数字表示系统是一个有限维的离散系统,输入输出数据需用有限维向量表示, 因此很多数学问题并不能由计算机直接处理. 这就需要我们对这些数学模型进行近似, 转化为所谓的“数值问题”.*

例如, 日常生活中的天气预报问题首先需要建立一个非常复杂的数学模型, 它往往是由大气动力学中的各种控制方程如连续性方程、运动方程、水汽方程和状态方程等等构成的微分方程组. 为求解该数学问题我们首先需要近似方法离散该无穷维问题, 将其转化为一个有限维问题, 如采用有限差分方法或有限元方法离散逼近. 其次, 离散之后得到的有限维系统可能是一个高维的线性或非线性代数方程组, 我们仍需设计有效的数值方去求解相应方程组. 最后, 根据设计的数值方法编制算法程序在计算机上实现并算出结果, 对天气情况进行预报.

事实上, 上述例子包含了数值分析中关心的两大类问题. 一类是含有(未知)函数的计算, 这里主要牵涉到方程中微分与积分的计算、微分方程的求解等. 处理的方法一般采用函数的数值逼近或离散化方法将原问题转化为数值问题. 另一类问题是代数问题的求解, 上例主要牵涉到线性或非线性代数方程组的数值方法.

数值分析也称为计算方法, 是数学与计算机的有机结合, 对应的数学分支称为计算数学. 随着计算机技术的飞速发展, 数学方法及计算已成为当今科学研究中不可缺少的手段. 通过建立数学模型, 应用数学理论和方法, 并结合计算机解决实际问题已成为十分普遍的研究模式. 目前, 数值分析方法已深入计算物理、计算力学、计算化学、计算生物学、计算经济学等各个领域. 因此, 近年来数值分析也被称为科学计算. 科学计算借助

*数值问题是指输入数据(即问题中自变量与原始数据)与输出数据之间函数关系的一个确定描述, 输入输出数据需用有限维向量表示 [8].

School of Math. NJU

计算机高速计算的能力, 融建模、算法、软件研制和计算模拟为一体, 是计算机实现其在高科技领域应用的必不可少的纽带和工具. 计算已不仅仅只是作为验证理论模型正确性的手段, 大量的事例表明它已成为重大科学发现的一种重要手段. 科学计算已经和理论研究与实验研究一起, 成为当今世界科学技术创新的主要方式, 也是当前公认的从事现代科学研究的第三种方法.

那么, 为了使一个数值方法在计算机上得到实现, 我们需要给出数值方法的一种**算法**, 它是算术和逻辑运算的完整描述, 按一定顺序执行这些运算, 经有限步把输入数据的每一个容许集转换成输出数据. 例如, 我们要求解非线性方程 $f(x) = x^5 + 2x^2 + 1 = 0$ 在区间 $[-2, -1]$ 上的根. Newton 迭代法就是求解该问题的一种数值方法(见附录B). 具体描述如下: 给定初值 x_0, 采用迭代公式

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = \frac{4x_k^5 + 2x_k^2 - 1}{5x_k^4 + 4x_k},\ k = 0, 1, 2, \cdots, \tag{1.1.1}$$

产生一个序列 $x_0, x_1, \cdots, x_n, \cdots$. 可按照某种判别准则在第 n 步停止, 输出 x_n 作为非线性方程的近似根. 具体到计算机上实现时, 设计相应的算法如下:

输入 初始近似值 $x_0 \in [-2, -1]$; 最大迭代次数 m, 容许误差限 ϵ.

输出 $x^5 + 2x^2 + 1 = 0$ 的近似根 p 或求解失败信息.

Step 1 $p_0 \leftarrow x_0$.

Step 2 对 $n = 1, \cdots, m$ 做 Step 3–4.

Step 3 $p \leftarrow \dfrac{4p_0^5 + 2p_0^2 - 1}{5p_0^4 + 4p_0}$.

Step 4 若 $|p - p_0| < \epsilon$, 则输出 (p), 停机, 否则 $p_0 \leftarrow p$.

Step 5 输出("方法失败");

停机.

算法中 "$\leftarrow$" 表示赋值. 若输出 "方法失败", 则表明迭代 m 次得到的 x_m, x_{m-1} 仍不满足 $|x_m - x_{m-1}| < \epsilon$.

这是一个求解特定的非线性方程根的算法, 我们可以通过输入任意非线性函数及其导数的定义来修改上面算法, 使其可以采用Netwon迭代法求解任意非线性方程的根(见附录B算法B.1).

设计一个好的数值方法(算法)应遵循以下基本原则: (1) 便于在计算机上实现; (2) 计算工作量尽量小; (3) 存贮量尽量小; (4) 问题的解与其近似解的误差小. 原则中的 (2) 和 (3) 保证了算法的高效性, (1) 保证了算法的可用性. 而 (4) 必须通过理论分析, 即收敛性、稳定性等数学理论保证, 并通过数值实验加以验证.

School of Math, NJU

1.2 误差的基本概念

1.2.1 误差来源

实际问题的精确解与数值计算所得到的近似解之间的差别被称为误差. 引起误差的因素很多, 主要有下面一些原因:

1. 模型误差与观测误差 在建模过程中, 欲将复杂的物理现象抽象并归结为数学模型, 往往需要忽略一些次要因素的影响, 对问题做某些必要的简化. 这样建立起来的数学模型实际上只是所研究的复杂客观现象的一种近似的描述, 它与真正客观存在的实际问题之间有一定的差别, 这种误差称为**模型误差**.

例如, 用 $x = A\cos\omega t + \varphi$ 来表示单摆运动, 计算在时刻 t 单摆的位移量 x. 但是这个方程就忽略了空气阻力等因素.

数学模型中常包含某些参量, 如质量、温度、电压等, 是由实验或观测得到的. 由于实验设备或者观测手段的限制, 得到的数据往往有一定误差, 该误差也被称为**观测误差**.

2. 方法误差 由实际问题建立的数学模型常常不能获得精确解, 必须用数值方法求近似解, 其误差称为**方法误差**. 它是真实结果(对于实际输入)与用给定数值方法经精确运算得到的结果之间的差. 由于计算机只能完成有限次算术运算和逻辑运算, 因此需将解题过程化为一系列有限的算术运算和逻辑运算. 常采用的策略包括将无穷维问题近似为有限维问题, 将无限过程近似为有限过程, 如用有限差分代替导数, 代数方程代替微分方程, 用有限和代替积分或无穷级数等.

例如, 微积分课程里介绍的泰勒多项式就是一种函数的近似计算公式. 事实上, 它只是取了展开级数的开头几项的部分和作为近似公式, 这样得到的近似值与准确值之间的截断误差就是由于采用截去余项的近似方法产生的方法误差.

又如, 求曲边梯形 $abBA$ (图 1.1) 的面积可表示为定积分:

$$S = \int_a^b f(x)\mathrm{d}x.$$

为近似计算上述积分, 我们将区间 $[a, b]$ 分割成 n 等分, 记 $x_i = a + ih$ $(i = 0, 1, 2, \cdots, n)$, 其中 $h = (b - a)/n$. 利用 $x = x_i (i = 1, 2, \cdots, n-1)$ 将曲边梯形 $abBA$ 分成 n 个小的曲边梯形, 然后再用小的矩形面积 $f(x_i)h (i = 0, 2, \cdots, n-1)$ 来近似地代替这些小的曲边梯形面积, 最终求出定积分的近似值. 即用

$$T = \frac{b-a}{n}\sum_{i=0}^{n-1} f(x_i)$$

作为 S 的近似值, 则产生误差 $S - T$, 它是由于把连续型问题离散化而产生的.

3. 计算误差 有了求解问题的算法后, 由于计算手段限制如计算机的字长有限, 原始数据在参与计算时会产生误差, 每一次运算又可能产生新的误差, 这种误差称为**计算误差**.

School of Math, NJU

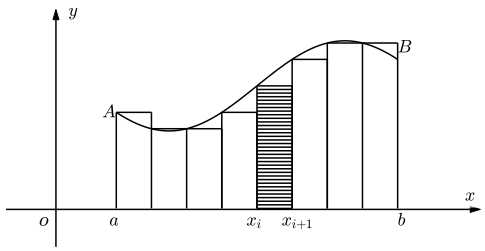


图 1.1 曲边梯形面积近似

例如, 无理数

$$\pi = 3.14159265\cdots,$$
$$\sqrt{2} = 1.41421356\cdots,$$

往往就需要按四舍五入近似成一定位数的有理数来代替, 如取 $\pi \approx 3.14159$. 由此引起的误差也被称为**舍入误差**. 该误差是用给定的算法经精确计算得到的结果与用同样的算法经有限精度得到的结果之差. 它是由实数的不精确表示以及对它们进行的操作引起的.

在对一个实际问题的数值模拟中, 上述类型误差往往会同时出现. 模型误差和观测误差属于原始误差, 主要是建模阶段产生的误差. 数值分析主要关心的是方法误差和舍入误差对模拟结果的影响.

1.2.2 绝对误差和相对误差

假设某一个量的准确值(真值)为 x, 其近似值为 $\bar{x}$. 我们称

$$e_{\bar{x}} = x - \bar{x} \tag{1.2.1}$$

为近似值 $\bar{x}$ 的**绝对误差**(简称**误差**). 一般情况下, 我们并不知道准确值 x, 因此无法准确计算 $e_{\bar{x}}$, 只能根据近似方法估计出绝对误差的大小范围. 例如, 无理数 $\mathrm{e} = 2.71828182\cdots$, 取 $\mathrm{e} = 2.718$, 则 $|\mathrm{e} - \bar{\mathrm{e}}| < 0.0003$.

也就是说, 可以指定一个正数 ε, 使

$$|e_{\bar{x}}| = |x - \bar{x}| \leqslant \varepsilon.$$

我们称 ε 为近似值 $\bar{x}$ 的一个**绝对误差界**.

要决定一个量的近似值的精确程度, 除了要看误差的大小外, 往往还要考虑该量本身的大小. 例如, 统计整个学校的人数时误差为 1 的意义要远小于统计一个班级的人数时误差为 1 的意义. 因此绝对误差还不足以刻画近似数的精确程度. 为此, 我们引入定义

$$r_{\bar{x}} = \frac{x - \bar{x}}{x}$$

School of Math. NJU

为 $\bar{x}$ 的**相对误差**. 和绝对误差一样, 在计算过程中一个量的准确值往往是不知道的, 因此常常将 $\bar{x}$ 的相对误差 $r_{\bar{x}}$ 定义为

$$r_{\bar{x}} = \frac{x - \bar{x}}{\bar{x}}.$$

同样, 相对误差一般也只能估计它的大小范围, 即指定一个正数 δ, 使得 $|r_{\bar{x}}| \leqslant \delta$. 我们称 δ 为 $\bar{x}$ 的一个**相对误差界**.

1.2.3 舍入误差与有效数字

1. 十进制与二进制数的表示 十进制以 10 为基数, 如 528.625 可详细写成为

$$528.625 = 5 \times 10^2 + 2 \times 10^1 + 8 \times 10^0 + 6 \times 10^{-1} + 2 \times 10^{-2} + 5 \times 10^{-3}.$$

若我们允许在小数点右边可以出现无穷多个数字, 则任何实数都可以以上述方式并在其前面加上符号来表示. 在二进制中, 仅使用两个数字 0 和 1. 用二进制我们也能详细写出一个具体的数. 例如

$$(1011.10101)_2 = 1\times2^3+0\times2^2+1\times2^1+1\times2^0+1\times2^{-1}+0\times2^{-2}+1\times2^{-3}+0\times2^{-4}+1\times2^{-5}.$$

这个数与十进制数 11.65625 相同. 为区分数 N 用什么样的基数, 可采用符号 $(N)_\beta$ 表示, 其中 β 代表基数. 例如 $(1011.10101)_2 = (11.65625)_{10}$.

注意计算机内部以二进制工作而以十进制方式与其用户交流, 因此这里牵涉到两种转换, 将十进制转换为二进制, 与将二进制转换回十进制. 这些转换由计算机完成. 由于计算机字长的限制, 不能对超过一定二进制位数的实数进行表示, 因此在转换时产生一些小的误差. 如 1/10 不能在任何二进制计算机中精确地存储, 因为它需要一个无穷的二进制表达式:

$$(0.1)_{10} = (0.00011001100110011\cdots)_2.$$

若把 0.1 读入一个 32 位的计算机, 然后把它的 40 位小数打印出来, 则得到下面的结果:

$$0.1000000014901161193847656250000000000000.$$

2. 舍入误差 一般说来, 一个实数的表示是无限的, 例如, $1/3 = 0.33333333\cdots$, $\mathrm{e} = 2.71828182\cdots$. 因此, 我们将一个实数 a 表示成无限小数的形式:

$$x = \pm x_0x_1\cdots x_m.x_{m+1}\cdots x_nx_{n+1}\cdots,$$

其中 $x_i(i = 0, 1, \cdots)$ 都是 $0, 1, 2, \cdots, 9$ 中的一个数字, 且 $m \neq 0$ 时, $x_0 \neq 0$.

通常, 我们用四舍五入的规则去取一个无限小数的近似值, 即若 x 的近似 $\bar{x}$ 取 $n-m$ 位小数时, 有

$$\bar{x} = \begin{cases} \pm x_0x_1\cdots x_m.x_{m+1}\cdots x_n, x_{n+1} \leqslant 4; \\ \pm(x_0x_1\cdots x_m.x_{m+1}\cdots x_n + 10^{-(n-m)}), x_{n+1} \geqslant 5. \end{cases}$$

School of Math, NJU

此时

$$|x-\bar{x}| \leqslant \frac{1}{2} \times 10^{-(n-m)}, \tag{1.2.2}$$

按此规律产生的误差称为**舍入误差**. 例如, $\mathrm{e} = 2.71828182\cdots$, 取三位小数时, $\bar{\mathrm{e}} = 2.718, |\mathrm{e}-\bar{\mathrm{e}}| < \frac{1}{2} \times 10^{-3}$; 取五位小数时, $\bar{\mathrm{e}} = 2.71828, |\mathrm{e}-\bar{\mathrm{e}}| < \frac{1}{2} \times 10^{-5}$.

3. 有效数字 假定有一个近似数

$$\bar{x} = \pm x_0 x_1 \cdots x_m . x_{m+1} \cdots x_n, \tag{1.2.3}$$

其中 $x_i (i = 0, 1, \cdots, n)$ 均是 $0, 1, \cdots, 9$ 的一个数字, 且 $m \neq 0$ 时, $x_0 \neq 0$. 若 $\bar{x}$ 的绝对误差满足 (1.2.2) 式, 且 x_s 是 $\bar{x}$ 的第一位(自左至右)非零数字, 则自 x_s 起到最右边的数字为止, 所有的数字都叫做 $\bar{x}$ 的**有效数字**, 并且说 $\bar{x}$ 是具有 $(n+1-s)$ 位有效数字的**有效数**.

例如无理数 $\mathrm{e} = 2.71828182\cdots$ 的近似数 $\bar{\mathrm{e}} = 2.718$ 具有四位有效数字, 其中 $2, 7, 1, 8$ 均为 $\bar{\mathrm{e}}$ 的有效数字. 又如 $x = 0.02365$ 的近似数 $\bar{x} = 0.024$ 是具有二位有数数字的有效数, 其中 $2, 4$ 为有效数字. 注意其中小数点后面第一个 0 不是有效数字. 另外, 有效数 12.3 与有效数 12.30 是不一样的, 后者末位的 0 也是有效数值, 因此有四位有效数字, 而前者只有三位有效数字.

关于有效数字和相对误差的关系, 我们有下面的定理.

定理 1.2.1. *若形如 (1.2.3) 的近似数 $\bar{x}$ 具有 $n+1-s$ 位有效数字, 则其相对误差有估计式*

$$\left|\frac{x-\bar{x}}{\bar{x}}\right| \leqslant \frac{1}{2x_s} \times 10^{-(n-s)},$$

其中 $x_s \neq 0$ 是 $\bar{x}$ 的第一位有效数字.

证明 首先, 若 $s = 0$, 此时 $x_0 \neq 0$, 由 (1.2.3) 知

$$|\bar{x}| \geqslant x_0 \times 10^m.$$

再由 (1.2.2) 式有

$$\left|\frac{x-\bar{x}}{\bar{x}}\right| \leqslant \frac{1}{x_0 \times 10^m} \times \frac{1}{2} \times 10^{-(n-m)} = \frac{1}{2x_0} \times 10^{-n}.$$

其次, 若 $s \neq 0, x_s \neq 0$, 此时 $m = 0$, 由 (1.2.3) 知

$$|\bar{x}| \geqslant x_s \times 10^{-s}.$$

再由 (1.2.2) 式有

$$\left|\frac{x-\bar{x}}{\bar{x}}\right| \leqslant \frac{1}{x_s \times 10^{-s}} \times \frac{1}{2} \times 10^{-n} = \frac{1}{2x_s} \times 10^{-(n-s)}. \qquad \square$$

School of Math, NJU

这个定理表明, 一个近似数的有效数字愈多, 其相对误差则愈小, 因而精确度愈高. 例如, 有效数 12.30 代表的近似数就比 12.3 代表的近似数的精确度高. 据定理 1.2.1, 前者的相对误差界为 $\frac{1}{2}\times 10^{-3}$, 而后者的相对误差界为 $\frac{1}{2}\times 10^{-2}$.

1.3 机器误差

1.3.1 规格化的科学记数法

在十进制中, 任何实数都可通过移动小数点和补充 10 的相应幂次使所有数字都在小数点右边并且第一位数字不是 0. 例如, $731.5029 = 0.7315029\times 10^3$, $-0.005123 = -0.5123\times 10^{-2}$. 一般而言, 一个非零实数 x 可采用如下规格化的科学记数法形式:

$$x = \pm r\times 10^n,$$

其中 $0.1\leqslant r<1$, n 为整数(正数、负数或零). 称数 r 为尾数, 称整数 n 为指数. r 和 n 都是以 10 为基数的数. 当然, 若 $x=0$, 则 $r=0$; 在其他所有情况下, 我们可以调节 n 使 r 处于给定范围内.

用完全同样的方法, 在二进制下, 一个实数可表示成

$$x = \pm q\times 2^m,$$

其中 $1/2\leqslant q<1$ (如果 $x\neq 0$), m 为整数. 同样称数 q 为尾数, 称整数 m 为指数. q 和 m 都是以 2 为基数的数.

1.3.2 计算机中数的表示

除二进制外, 在计算机科学中, 还广泛采用八进制和十六进制系统表示数的方法, 它们的基数分别为 8 和 16.

一般地, 一个 p 进制数 x 可以表示成

$$x = \pm p^J\sum_{k=1}^{t} d_k p^{-k}, \tag{1.3.1}$$

其中, $d_k(k=1,2,\cdots,t)$ 都是 $0,1,\cdots,p-1$ 中的一个数字. (1.3.1) 式或写成

$$x = \pm r\times p^J, \tag{1.3.2}$$

其中

$$r = \sum_{k=1}^{t} d_k p^{-k} = (0.d_1d_2\cdots d_t)_p.$$

School of Math, NJU

我们称 r 为数 x 的**尾数**(其值小于 1). 自然数 t 为计算机的**字长**, 它表示数 x 的尾数的位数. J 是整数, 称为数 x 的**阶**, 它用来确定该数的小数点的位置.

在各种计算机中, 有各自规定的字长 t, 以及阶 J 的范围: $-L \leqslant J \leqslant U$($L$ 和 U 为正整数或零). L, U的大小表明计算机中表示的数的范围大小. 如果阶 J 可以变化, 我们则称 (1.3.1) 或 (1.3.2) 为**浮点**表示. 为避免发生不同的浮点表示, 我们将浮点表示规格化, 规定在浮点表示中尾数的第一位数字 d_1 非零. 这种浮点表示的数, 称为**规格化浮点数**. 显然, 对十进制规格化浮点数, 其尾数满足: $0.1 \leqslant r < 1$; 对二进制规格化浮点数, 其尾数满足: $\frac{1}{2} \leqslant r < 1$.

按 (1.3.1) 规定的浮点数的全体组成的集合记作 $\mathcal{F}$, 再假定当 $x \neq 0$ 时, $d_1 \neq 0$, 则称 $\mathcal{F}$ 为**规格化浮点数系**, 它的基数为 p. 规格化浮点数系 $\mathcal{F}$ 是一个离散的有限集合, 它由 4 个整数 (p, t, L, U) 表示. 其中 p 代表基数或底数, t 代表精度, $[-L, U]$ 代表指数范围. 可以证明: 规格化浮点数系 $\mathcal{F}$ 中共有 $2(p-1)p^{t-1}(L+U+1)+1$ 个浮点数.

规格化浮点数系中的最小正数称为**下溢值**: $UFL = p^{-L-1}$, 其尾数的首位为 1, 其余位为零, 指数取最小的可能值. 最大的浮点数称为**上溢值**: $OFL = p^U(1-p^{-t})$, 其尾数的每一位都为 $p-1$, 指数取最大的可能值. 任何超过 OFL 的数或小于 UFL 的正数都不能用浮点数系统表示. 在浮点范围内, 浮点数并不是均匀分布的, 但在 p 的相同幂次间是等距的, 即同阶的浮点数之间尾数相差 p^{-t}.

例如, 若 $p = 2, t = 3, L = 2, U = 2$, 则相应的规格化浮点数系 $\mathcal{F}$ 中的规格化浮点数具有形式

$$x = \pm r \times 2^J = \pm(d_1 \times 2^{-1} + d_2 \times 2^{-2} + d_3 \times 3^{-3}) \times 2^J,$$

其中当 $x \neq 0$ 时, $d_1 = 1, d_2, d_3$ 为 $0, 1$ 中的一个数字, $J = -2, -1, 0, 1, 2$. 因此, $\mathcal{F}$ 中有 41 个浮点数如下:

$J=-2$	$J=-1$	$J=0$	$J=1$	$J=2$
$\pm\frac{1}{8}$	$\pm\frac{1}{4}$	$\pm\frac{1}{2}$	± 1	± 2
$\pm\frac{5}{32}$	$\pm\frac{5}{16}$	$\pm\frac{5}{8}$	$\pm\frac{5}{4}$	$\pm\frac{5}{2}$
$\pm\frac{3}{16}$	$\pm\frac{3}{8}$	$\pm\frac{3}{4}$	$\pm\frac{3}{2}$	± 3
$\pm\frac{7}{32}$	$\pm\frac{7}{16}$	$\pm\frac{7}{8}$	$\pm\frac{7}{4}$	$\pm\frac{7}{2}$

以及数 0. 其中上溢值 OFL 为 7/2, 下溢值 UFL 为 1/8.

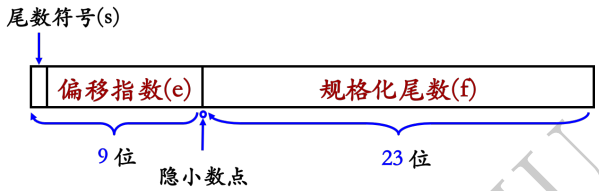
当计算机中存储二进制数时, 我们对(1.3.1)式稍加修改. 假设把二进制数首位数字 1 恰好移到二进制小数点的左边. 在这种情况下, 尾数部分表达式改为

$$r = (1.f)_2.$$

School of Math, NJU

显然 $1 \leqslant r < 2$. 现在仅把 $(.f)_2$ 以计算机的形式存储, 从而利用实际不存储二进制首位数字 1 来节省一位空间. 上述存储方式称为**左移规格化二进制数**.

我们来考虑一台 32 位的计算机中单精度实数的浮点表示方法. 在表示一个非零实数 $x = \pm q \times 2^m$ 时用下面的方式分配组成一个字的二进制位: 其中, 实数 x 的符号占 1



位, 偏移指数(整数 e)占 8 位, 规格化尾数(实数 f)占 23 位.

实际上, 由于采用左移规格化浮点数, 这台计算机的浮点数有 24 位尾数. 因此, 非零规格化的二进制数是一个位串, 其数值被解码如下:

$$x = (-1)^s q \times 2^m, \tag{1.3.3}$$

其中 $q = (1.f)_2$, 并且 $m = e - 127$. 此外, s 表示 x 的符号(正数用 0, 负数用 1 表示). 这里 $1 \leqslant q < 2$, 并且 q 中最高有效数字为 1 且不显式存储. m 是 8 位偏移指数, 而 f 是实数 x 的 23 个小数部位, 与隐含的首位 1 一起得到有效数字字段.

如果一个实数能按(1.3.3)的形式, 用占 8 位的 $|m|$ 和占 23 位的 q 表示, 则它是32位计算机中的一个机器数. 即它能在这个特殊的计算机中被精确表示. 大多数实数在该计算机中不能被精确表示, 像这样的数作为输入数据或作为计算结果出现, 那么在用机器数尽可能精确表示它的时候就会产生不可避免的误差.

$|m|$ 表示不超过 8 位的限制意味着

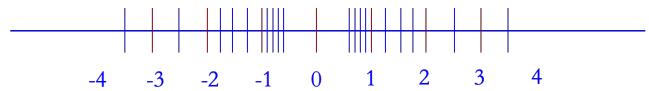
$$0 \leqslant e \leqslant (11111111)_2 = 2^8 - 1 = 255,$$

其中 $e = 0$ 和 $e = 255$ 为像 $\pm 0, \pm\infty$ 和 NaN (非数字) 这样的特殊情况保留. 进一步, 由 $m = e - 127$, 有 $-126 \leqslant m \leqslant 127$. 于是, 我们得该计算机中的下溢值为 $2^{-126} \approx 1.2 \times 10^{-38}$, 上溢值为 $(2 - 2^{-23})2^{127} \approx 3.4 \times 10^{38}$. 尾数部分要求不超过 23 位意味着我们的机器数大概有 7 位十进制小数的有限精度, 这是因为尾数的最低有效位表示 $2^{-23}(\approx 1.2 \times 10^{-7})$个单位.

在二进制计算机中, 浮点数分布相当不均匀, 大部分集中在 0 附近. 这是因为, 在计算机中仅仅存在有限个机器数, 且在 2 的相邻幂次间总是存在相同个数的机器数. 而 2 的幂之间的间隙在 0 附近较小, 在远离 0 处较大. 这产生了浮点数的不均匀分布, 在原点附近密度较高.

School of Math, NJU

例 1.3.1 (浮点系统)**.** 下图是一个浮点系统, 其中 $p=2, t=2, L=1, U=1$, 图中标出了系统中所有的 25 个浮点数. 对这个系统最大数为 $OFL=(1.11)_2\times 2^1=(3.5)_{10}$, 最小的规范化正数为 $UFL=(1.00)_2\times 2^{-1}=(0.5)_{10}$.



1.3.3 机器舍入

能在某个浮点系统 $\mathcal{F}$ 中精确表示的实数称为**机器数**. x 不能用浮点数精确表示, 则用某个"邻近"的浮点数近似. 记实数 x 浮点近似为 $fl(x)$. 选择一个邻近的浮点数 $fl(x)$ 近似一个给定实数 x 的过程称为**舍入**, 由于这种近似产生的误差称为**舍入误差**. 目前的计算机主要采用最近舍入原则, 即取 $fl(x)$ 为与 x 最接近的那个机器数. 在距离相同时, 采用舍入到最近的偶数, 即若实数正好在其左右两个机器数的中间时, 选择尾数为偶数的机器数.

例如, 对十进制数, 在机器数值范围内的任何一个实数可以规格化为

$$x=\pm r\times 10^J, \tag{1.3.4}$$

其中

$$r=0.d_1\cdots d_t d_{t+1}\cdots d_n,\quad 0\leqslant d_i\leqslant 9\ (i=1,\cdots,n),\ d_1>0.$$

若 $d_{t+1}=5$, 且 $d_i=0, i>t+1$ 时, 则取

$$\bar{r}=\begin{cases} 0.d_1\cdots d_t, & \text{若 } d_t \text{ 为偶数;}\\ 0.d_1\cdots d_t+10^{-t}, & \text{若 } d_t \text{ 为奇数,}\end{cases}$$

否则取

$$\bar{r}=\begin{cases} 0.d_1\cdots d_t, & \text{若 } 0\leqslant d_{t+1}\leqslant 4;\\ 0.d_1\cdots d_t+10^{-t}, & \text{若 } d_{t+1}\geqslant 5.\end{cases}$$

于是

$$fl(x)=\pm\bar{r}\times 10^J. \tag{1.3.5}$$

从而有

$$\begin{aligned}\left|\frac{fl(x)-x}{x}\right| &= \left|\frac{\pm\bar{r}\times 10^J-(\pm r)\times 10^J}{\pm r\times 10^J}\right|\\ &= \left|\frac{\bar{r}-r}{r}\right|.\end{aligned}$$

School of Math, NJU

据(1.2.2)式, 并注意到 $r \geqslant 10^{-1}$, 便有

$$\left|\frac{\bar{r}-r}{r}\right| \leqslant \frac{1}{2r} \times 10^{-t} \leqslant \frac{1}{2} \times 10^{-t+1} = 5 \times 10^{-t},$$

即有

$$\left|\frac{fl(x)-x}{x}\right| \leqslant 5 \times 10^{-t}.$$

令

$$\frac{fl(x)-x}{x} = \varepsilon,$$

则得到关系式

$$fl(x) = x(1+\varepsilon), \quad |\varepsilon| \leqslant 5 \times 10^{-t}. \tag{1.3.6}$$

对于二进制系统数

$$x = \pm q \times 2^J, \tag{1.3.7}$$

其中 $\frac{1}{2} \leqslant q < 1$, 且

$$q = 0.d_1 \cdots d_t d_{t+1} \cdots d_n, \quad d_i = 0 \text{ 或 } 1 (i = 2, \cdots, n),\ d_1 = 1.$$

若 $d_{t+1} = 1$, 且 $d_i = 0, i > t+1$ 时, 则取

$$\bar{q} = \begin{cases} 0.d_1 \cdots d_t, & \text{若 } d_t = 0; \\ 0.d_1 \cdots d_t + 2^{-t}, & \text{若 } d_t = 1, \end{cases}$$

否则取

$$\bar{q} = \begin{cases} 0.d_1 \cdots d_t, & \text{若 } d_{t+1} = 0; \\ 0.d_1 \cdots d_t + 2^{-t}, & \text{若 } d_{t+1} = 1, \end{cases}$$

于是

$$fl(x) = \pm \bar{q} \times 2^J. \tag{1.3.8}$$

类似于十进制系统, 我们有

$$fl(x) = x(1+\varepsilon), \quad |\varepsilon| \leqslant 2^{-t}. \tag{1.3.9}$$

若一台计算机用 p 作基数并且其浮点数的尾数有 t 位, 则

$$fl(x) = x(1+\varepsilon), \varepsilon \leqslant \varepsilon_{\text{Mach}}.$$

其中, 在最近舍入情况下 $\varepsilon_{\text{Mach}} = \frac{1}{2}p^{1-t}$. $\varepsilon_{\text{Mach}}$ 被称为**机器精度**(单位舍入误差), 它说明了用浮点系统表示一个非零实数 x 的最大可能的相对误差, 即

$$\left|\frac{fl(x)-x}{x}\right| \leqslant \varepsilon_{\text{Mach}}.$$

School of Math, NJU

虽然机器精度和下溢值都是很小的正数, 但二者不同. 机器精度 $\varepsilon_{\text{Mach}}$ 是由浮点系统尾数域的位数决定的, 而下溢值 UFL 则是由指数域的位数决定的. 在所有浮点系统中

$$0 < UFL < \varepsilon_{\text{Mach}} < OFL.$$

1.3.4 浮点运算

假定给定实数 x 和 y 的浮点表示为 $fl(x)$ 和 $fl(y)$, 又假定符号 $\oplus$, $\ominus$, $\otimes$, $\oplus$ 分别代表机器加、减、乘和除的操作. 则

$$\begin{aligned} x \oplus y &= fl(fl(x) + fl(y)), \quad x \ominus y = fl(fl(x) - fl(y)), \\ x \otimes y &= fl(fl(x) \times fl(y)), \quad x \oplus y = fl(fl(x) \div fl(y)). \end{aligned}$$

这里假设首先对两个机器数之间进行精确的运算, 然后对结果进行规格化和舍入, 最后, 以计算机字的形式存储在存储器中. 浮点数计算时, 对加减法, 首先要**对阶**. 即将两数的小数点对齐, 使它们的阶相等. 对阶的方法是, 把阶小的数的尾数右移, 每移一位其阶就加“1”直到两数的阶相等. 然后, 将对阶后的两数相加(或相减). 需要注意的是作浮点运算时, 通常的运算规律不再成立. 另外, 在作乘法运算时, 不必对阶.

例 1.3.2. 假设所使用的计算机是采用最近舍入的方法, 且 $t = 5, p = 10$. 若 $x = 0.51752 \times 10^{-5}$, $y = 0.45123 \times 10^{-3}$, 则

$$\begin{aligned} x \oplus y &= fl(0.0051752 \times 10^{-3} + 0.45123 \times 10^{-3}) \\ &= 0.45641 \times 10^{-3}. \end{aligned}$$

例 1.3.3. 假设在采用最近舍入法的计算机($p = 10, t = 5$)上求数 $x = 0.423578$ 与 $y = 0.423572$ 的差, 则有

$$\begin{aligned} fl(x) \ominus fl(y) &= 0.42358 \times 10^0 \ominus 0.42357 \times 10^0 \\ &= fl(0.00001) = 0.10000 \times 10^{-4}. \end{aligned}$$

$fl(x)$ 的相对误差是

$$\frac{x - fl(x)}{fl(x)} \approx -0.47217 \times 10^{-5},$$

$fl(y)$ 的相对误差是

$$\frac{y - fl(y)}{fl(y)} \approx 0.47218 \times 10^{-5}.$$

但 $x - y = 0.000006$, $fl(x) \ominus fl(y)$ 的相对误差是

$$-\frac{0.000004}{0.00001} = -0.4.$$

$fl(x)$ 与 $fl(y)$ 很接近, $fl(x) \ominus fl(y) = 0.00001$ 只有一位有效数字, 产生了精度丢失. $fl(x) \ominus fl(y)$ 的相对误差约超过参与运算的近似数的相对误差的 10^5 倍.

School of Math, NJU

当两个几乎相等的同号数相减时, 精度的损失称为**相减相消**. 作为一个法则, 我们在设计数值算法时应避免用几乎相等同号量的减法, 这种减法将危及精度.

例 1.3.4. 求一元二次方程

$$ax^2 + bx + c = 0, \quad a \neq 0$$

的两个根 x_1, x_2 的公式是

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

和

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}.$$

若 $b > 0$ 且 $b^2 \gg 4ac \geqslant 0$, 计算 x_1 则会产生相减相消. 为了避免相减相消, 我们可将分子有理化, 把二次公式改写成

$$x_1 = \frac{-2c}{b + \sqrt{b^2 - 4ac}}.$$

现在, 我们的问题是: x 接近同号数 y 时, 在减法 $x - y$ 中确切地丢失了多少二进制位? 准确答案依赖于 x 和 y 的精确值. 下面的定理给出一个界.

定理 1.3.1 (精度丢失定理)**.** *若 x 和 y 是正的规格化二进制浮点数满足 $x > y$, 且 $2^{-q} \leqslant 1 - \dfrac{y}{x} \leqslant 2^{-p}$, 则在减法 $x - y$ 中丢失至多 q 个、至少 p 个有效的二进制位.*

证明 设 x 和 y 的规格化二进制浮点形式是

$$x = r \times 2^n (1/2 \leqslant r < 1), \quad y = s \times 2^m (1/2 \leqslant s < 1).$$

因为 x 大于 y, 所以在执行 x 减 y 之前, 计算机必须对 y 移位以便它们有相同的阶(指数). 所以, 必须把 y 写成 $y = (s \times 2^{m-n}) \times 2^n$. 于是我们有

$$x - y = (r - s \times 2^{m-n}) \times 2^n.$$

这个数的尾数满足

$$r - s \times 2^{m-n} = r\left(1 - \frac{s \times 2^m}{r \times 2^n}\right) = r\left(1 - \frac{y}{x}\right) < 2^{-p}.$$

为使 $x - y$ 的计算机表示规格化, 至少需要向左移动 p 位. 因此至少有 p 个虚假的 0 被添加到尾数的右端, 这意味着至少丢失了 p 个二进制的精度.

同理有

$$r - s \times 2^{m-n} > 2^{-q-1}.$$

于是, 为使 $x - y$ 的计算机表示规格化, 至多需要向左移动 q 位. 这意味着至多丢失了 q 个二进制的精度. □

School of Math, NJU

1.3.5 数值运算的简单原则

为了减少数值运算中的舍入误差, 计算时通常需要注意一些原则. 例如, 前面提到的应尽量避免同号相近数相减, 防止精度丢失. 其次, 我们还应避免除数绝对值很小的除法, 其计算结果的误差可能会很大, 甚至出现计算溢出的情形. 另外, 数值运算时我们还需注意以下两点.

(1) 注意运算次序

在计算机中, 由于字长的限制, $(a+b)+c$ 可能不等于 $a+(b+c)$, 因为两者的舍入误差可能不同.

例 1.3.5. 在 3 位十进制的限制下, 计算

$$A = 100 + \delta_1 + \delta_2 + \cdots + \delta_{100},$$

其中 $0.1 \leqslant \delta_i \leqslant 0.3, i = 1, 2, \cdots, 100$.

解 如果自左到右相加, 则每个 δ_i 被 100 吃掉, A 的 3 位近似值为 100; 如果先把所有 δ_i 加起来再加 100, 则

$$110 = 100 + 0.1 \times 100 \leqslant A \leqslant 100 + 0.3 \times 100 = 130. \qquad \square$$

由此例可看出一个简单原则: 在多个数求和时, 如果被加数的绝对值之间差异较大, 且包含许多绝对值较小的数, 则应按绝对值从小到大的次序相加.

(2) 简化计算步骤

每次算术运算都可能产生舍入误差, 因此, 如能通过算法的改进减少运算次数, 特别是减少除法的运算次数, 则舍入误差的积累一般可能下降, 还能节省计算机的执行时间.

例 1.3.6. 求多项式

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

的值.

解 多项式是一个表达式, 但由它导出结果的过程有很大的随意性, 它不是一个算法而是一个计算问题. 若直接计算 $a_k x^k$ 再逐项相加, 则确定一种算法:

$$\begin{cases} A_0 = a_0, \\ A_k = a_k x^k \quad (k = 1, 2, \cdots, n), \\ P_n(x) = A_0 + A_1 + \cdots + A_n. \end{cases}$$

这个算法有明显的缺点, 比如 x 幂的重复计算, 它共需做 $1 + 2 + \cdots + n = \dfrac{n(n+1)}{2}$ 次乘法和 n 次加法.

School of Math, NJU

若用著名的秦九韶算法或称 Horner 算法

$$\begin{cases} S_n = a_n, \\ S_k = xS_{k+1} + a_k \quad (k = n-1, \cdots, 2, 1, 0), \\ P_n(x) = S_0 \end{cases} \tag{1.3.10}$$

求 $P_n(x)$ 的值只需做 n 次乘法和 n 次加法. □

1.4 算法的稳定性分析

1.4.1 算法的数值稳定性

定义 1.4.1. 一个算法如果输入数据有误差, 而在计算过程中舍入误差不增长, 则称此算法是数值稳定的, 否则, 算法称为不稳定的.

通俗地讲, 若一个算法数值计算过程某个阶段所产生的小误差在随后阶段中被放大从而严重降低了全部计算的精度, 则我们说这个数值过程是不稳定的.

例 1.4.1. 计算积分 $y_n = \int_0^1 \frac{x^n}{x+6}\mathrm{d}x (n = 0, 1, \cdots, 8)$ 可利用递推公式

$$y_n + 6y_{n-1} = \frac{1}{n},$$

其中 $y_0 = \int_0^1 \frac{1}{x+6}\mathrm{d}x = \ln\frac{7}{6}$.

解 利用递推公式

$$y_n = \frac{1}{n} - 6y_{n-1} \qquad (n = 0, 1, \cdots, 8), \tag{1.4.1}$$

计算 y_n, 且算到小数点后 4 位.

由于初始值 $y_0 = \ln\frac{7}{6} \approx 0.1542 = \tilde{y}_0$, 用 $\tilde{y}_0$ 近似 y_0, 误差为 $\varepsilon_0 = y_0 - \tilde{y}_0$. 由 (1.4.1) 得

$$\tilde{y}_n = \frac{1}{n} - 6\tilde{y}_{n-1} \qquad (n = 0, 1, \cdots, 8). \tag{1.4.2}$$

计算结果如表 1.1第三列所示. 表中 y_n 是误差小于 $\frac{1}{2} \times 10^{-8}$ 的积分近似值, 从表中看到 $\tilde{y}_n$ 的误差很大, 在 $n = 5$ 时就已完全失真, 因为 $y_n > 0$ 而 $y_5 < 0$ 显然不对. 这里计算公式是正确的, 只是因为初始近似 $\tilde{y}_0$ 有误差 ε_0, 其满足

$$|\varepsilon_0| = |y_0 - \tilde{y}_0| < \frac{1}{2} \times 10^{-4}.$$

School of Math. NJU

表 1.1 算法稳定性示例

n	y_n	$\tilde{y}_n$	$\bar{y}_n$
0	0.15415068	0.1542	0.1542
1	0.07509592	0.0748	0.0751
2	0.04942447	0.0512	0.0494
3	0.03678649	0.0261	0.0368
4	0.02928106	0.0934	0.0293
5	0.02431366	-0.3604	0.0243
6	0.02078469	2.3291	0.0208
7	0.01814902	-13.8317	0.0182
8	0.01610588	83.1152	0.0161

令 $\varepsilon_n = y_n - \tilde{y}_n$, 由式 (1.4.1) 减去 (1.4.2) 得

$$\varepsilon_n = -6\varepsilon_{n-1} = \cdots = (-6)^n\varepsilon_0, \quad |\varepsilon_n| \geqslant 6^n|\varepsilon_0|$$

它表明误差增长很快, 故算法 (1.4.1) 是不稳定的.

若将公式 (1.4.1) 改为

$$y_{n-1} = \frac{1}{6}\left(\frac{1}{n} - y_n\right) \qquad (n = 8, 7, \cdots, 1), \tag{1.4.3}$$

取

$$\bar{y}_8 = 0.0161, \quad \bar{\varepsilon}_8 = |y_8 - \bar{y}_8| < \frac{1}{2} \times 10^{-4},$$

由式 (1.4.3) 计算出

$$\bar{y}_{n-1} = \frac{1}{6}\left(\frac{1}{n} - \bar{y}_n\right) \qquad (n = 8, 7, \cdots, 1),$$

结果如表 1.1 最后一列所示. 此时

$$\bar{\varepsilon}_{n-1} = -\frac{1}{6}\bar{\varepsilon}_n, \quad \bar{\varepsilon}_{8-k} = (-1)^k\left(\frac{1}{6}\right)^k\bar{\varepsilon}_8,$$

误差是逐步递减的. 故用算法 (1.4.3) 计算是数值稳定的. □

在数值计算中数值不稳定的算法是不能使用的.

1.4.2 病态问题与条件数

数值稳定性是针对算法而言, 对数值问题本身如果输入数据微小变动引起输出数值(即问题的解)很大扰动(误差), 就是病态问题, 它是数学问题本身决定的, 与数值问题的求解算法无关.

School of Math, NJU

例 1.4.2. 求方程组 $\begin{cases} ax+y=1 \\ x+ay=0 \end{cases}$ 的解.

解 当 $a=1$ 时, 系数矩阵奇异, 解不存在.

当 $a \neq 1$ 时解为

$$x=\frac{a}{a^2-1}, \quad y=\frac{-1}{a^2-1}.$$

若输入数据 a 有误差, 且 $a \approx 1$, 则解的误差很大. 例如 $a=0.999$ 时的解 $y=500.25$, 若 a 有误差 0.0001 即 $\bar{a}=0.9991$, 则解 $\tilde{y} \approx 555.81$, 解误差 $\tilde{y}-y \approx 55.56$. 所以在此例中若 $a \approx 1$ 则问题是病态的, 这与选用何种求解方法无关. □

如何判断问题是否病态呢? 通常可用条件数来衡量. 下面以计算函数 $f(x)$ 的值为例给出条件数的定义

$$\begin{aligned} C_p &= \operatorname{cond}(f(x)) \\ &:= \lim_{\Delta x \to 0} \left| \frac{f(x+\Delta x)-f(x)}{f(x)} \Big/ \frac{\Delta x}{x} \right| \\ &= \left| \frac{x f'(x)}{f(x)} \right|, \end{aligned} \tag{1.4.4}$$

它是计算函数 $f(x)$ 的相对误差 $\left| \frac{f(x+\Delta x)-f(x)}{f(x)} \right|$ 与 x 相对误差之比的极限. 根据定义相对误差越大条件数 C_p 也越大, 病态越严重, 通常 $C_p \gg 1$ 就认为问题病态.

上例中 $y=\frac{1}{1-a^2}$ 是 a 的函数. 利用式 (1.4.4) 结果, 由于 $y'(a)=-\frac{2a}{(1-a^2)^2}$, 故

$$C_p = \left| \frac{a y'(a)}{y(a)} \right| = \frac{2a^2}{1-a^2}.$$

当 $a=0.999$ 时 $C_p \approx 999$, 故问题是病态的.

习 题 一

1. 应用梯形公式

$$T=\frac{b-a}{2}(f(a)+f(b))$$

计算积分

$$I=\int_0^1 e^{-x}\mathrm{d}x$$

的近似值, 在整个计算过程中按四舍五入规则取五位小数. 计算中产生的误差的主要原因是离散或是舍入? 为什么?

2. 证明: 若 a 按四舍五入规律舍入到具有 $n-m$ 位小数的近似数 $\bar{a}$, 则

$$|a-\bar{a}| \leqslant \frac{1}{2} \times 10^{-(n-m)}.$$

School of Math, NJU

3. 证明: 若近似数 $\bar{a} = \pm a_0 a_1 \cdots a_m.a_{m+1} \cdots a_n$ 的相对误差有估计式

$$\left|\frac{a-\bar{a}}{\bar{a}}\right| \leqslant \frac{1}{2(a_s+1)} \times 10^{-(n-s)},$$

其中 $a_s \neq 0$ 是 $\bar{a}$ 的第一位有效数字, 则 $\bar{a}$ 至少具有 $n+1-s$ 位有效数字.

4. 设 $\bar{x} = 23.3123, \bar{y} = 23.3122$, 且 $|e_{\bar{x}}| \leqslant \frac{1}{2} \times 10^{-4}, |e_{\bar{y}}| \leqslant \frac{1}{2} \times 10^{-4}$. 问差 $\bar{x} - \bar{y}$ 最多有几位有效数字?

5. 当 $x(>0)$ 很大时, 如何计算 (1) $\arctan(x+1) - \arctan x$; (2) $\ln(x - \sqrt{x^2-1})$; (3) $\dfrac{\tan x}{x - \sqrt{x^2-1}}$, 可使其误差较小.

6. 试计算方程

$$x^2 + 78x + 1 = 0$$

的两个根 ($x_1 = -0.0128226\cdots, x_2 = -77.9871773\cdots$), 整个计算过程中取四位小数, 要求所求得近似根尽可能精确.

7. 在 $p = 10, t = 4, L = U = 5$ 的采用舍入近似的假想计算机上, 将下列实数写成相应的规格化浮点数:

(1) -315.37; (2) 0.0074;

(3) $25\frac{2}{3}$; (4) $52\frac{1}{3}$.

8. 证明: 规格化浮点数系 F 中共有

$$2(p-1)p^{t-1}(L+U+1)+1$$

个浮点数.

9. 假设在 $p = 10, t = 4, L = U = 5$ 的采用舍入近似的假想计算机上作浮点运算. 设 $x = 0.6436 \times 10^{-4}, y = 0.8321 \times 10^{-3}$,求$x \oplus y$和$x \ominus y$.

10. 设字长 $t = 8$, 以及

$$x = 0.32282167 \times 10^{-4},$$
$$y = 0.45367549 \times 10^{2},$$
$$z = -0.45368822 \times 10^{2}.$$

求 $fl(fl(x+y)+z), fl(x+fl(y+z))$ 和 $x+y+z$.

上机实习题

1. 设计算法找出你所使用的计算机的机器精度、下溢值和上溢值. 编程验证所设计算法的正确性.

School of Math, NJU

2. 设计算法计算 $y = x - \sin x$, 使得有效位的丢失最多 1 位. 编程验证所设计算法的正确性.

3. 计算积分 $y_n = \int_0^1 x^n \mathrm{e}^x \mathrm{d}x (n \geqslant 0)$, 可由分部积分法得

$$y_{n+1} = \mathrm{e} - (n+1) y_n.$$

编程验证按上述公式设计的算法不稳定.

4. 考虑由 $\begin{cases} x_0 = 1, x_1 = 1/3, \\ x_{n+1} = \dfrac{13}{3} x_n - \dfrac{4}{3} x_{n-1}, \end{cases}$ 归纳定义的实数序列实现的算法是否稳定? 如果将初值改为 $x_0 = 1, x_1 = 4$, 数值稳定吗?

School of Math, NJU

附录 A 非线性方程的迭代解法

一、不动点迭代法

对一般的非线性方程

$$f(x) = 0, \tag{A.0.1}$$

我们往往采用迭代法求解. 迭代法是一个逐步逼近过程, 为此, 我们首先将原方程转化为同解的问题:

$$x = g(x). \tag{A.0.2}$$

这样把原问题的求解转化为寻找 g 的不动点. 为了求 g 的不动点, 我们选取一个初始近似值 x_0, 令

$$x_k = g(x_{k-1}), \qquad k = 1, 2, \cdots, \tag{A.0.3}$$

以产生序列 $\{x_k\}$. 如果 $\lim\limits_{k\to\infty} x_k = p$, 且 $g(x)$ 连续, 则对 (A.0.3) 两边同时取极限得

$$p = \lim_{k\to\infty} x_{k+1} = \lim_{k\to\infty} g(x_k) = g(p).$$

则 p 是 g 的一个不动点. 因此, p必为方程 (A.0.1) 的一个解.

这一类迭代法称为不动点迭代法, 或 Picard 迭代法. $g(x)$ 又称为迭代函数.

算法 A.1. 用不动点迭代求方程 $x = g(x)$ 的一个解.

输入 初始值 x_0, 误差容限 TOL; 最大迭代次数 m.

输出 近似解 p, 或失败信息.

step 1 对 $k = 1, 2, \cdots, m$ 做 step 2–3.

step 2 $p \leftarrow g(x_0)$.

step 3 若 $|p - x_0| <$ TOL 则输出 (p), 停机;

否则, $x_0 \leftarrow p$.

step 4 输出('Method failed'); 停机

在第3步中, 迭代终止准则可用

$$\frac{|p - x_0|}{|p|} < \text{TOL}.$$

迭代函数的选择必须遵循如下基本原则: 首先, 必须保证 Picard 迭代产生的迭代序列 $x_0, x_1, \cdots, x_k, \cdots$ 在 $g(x)$ 的定义域中, 以使迭代过程不致于中断; 其次, 要求迭代序列 $\{x_k\}$ 收敛且尽可能收敛得快.

School of Math. NJU

二、Newton-Raphson方法

考虑非线性方程 (A.0.1). 将 $f(x)$ 在初值处作 Taylor 展开

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \cdots .$$

取展开式的线性部分作为 $f(x)$ 的近似, 有

$$f(x_0) + f'(x_0)(x - x_0) \approx 0$$

若 $f'(x_0) \neq 0$, 则有

$$x = x_0 - \frac{f(x_0)}{f'(x_0)}$$

记为 x_1. 如图 A.1 所示, x_1 为过 $(x_0, f(x_0))$ 的切线与 x 轴的交点.

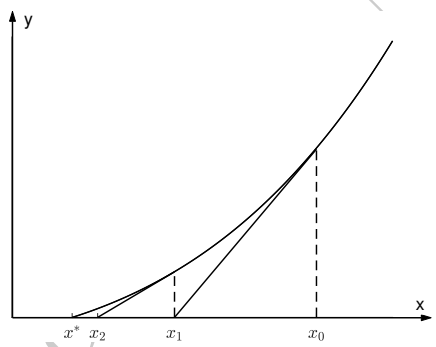


图 A.1　Newton 迭代法

类似, 我们可以得到

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)}.$$

这样一直下去, 我们可以得到迭代序列 $x_3, x_4, \cdots$.

上述迭代法称为 Newton-Raphson 方法(或简称 Newton 法), 是解非线性方程的最著名的和最有效的数值方法之一.

该方法从不动点迭代的角度可解释如下. 若 $f'(x) \neq 0$, 令

$$g(x) = x - \frac{f(x)}{f'(x)}, \tag{A.0.4}$$

则方程 $f(x) = 0$ 和 $x = g(x)$ 是等价的.

我们选取 (A.0.4) 为迭代函数. 据 (A.0.3) 式, Picard 迭代为

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \qquad k = 0, 1, 2, \cdots . \tag{A.0.5}$$

我们称 (A.0.5) 为 Newton 迭代公式, 称 $\{x_k\}$ 为 Newton 序列.

School of NJU

算法 A.2. 用 Newton 迭代法求方程 $f(x)=0$ 的一个解.

输入 x_0, TOL1, TOL2, m

$v \leftarrow f(x_0)$

输出 $0, x_0, v$

if $|v|$ <TOL2 then stop

for $k=1$ to m do

$x_1 \leftarrow x_0 - v/f'(x_0)$

$v \leftarrow f(x_1)$

输出 k, x_1, v

if $|x_1 - x_0|$ <TOL1 or $|v| <$ TOL2 then stop

$x_0 \leftarrow x_1$

enddo

注 B.1 迭代终止准则还可以用

$$\frac{|x_1 - x_0|}{|x_1|} < \text{TOL1}$$

或者

$$\frac{|x_1 - x_0|}{|x_1|} < \text{TOL1} \quad \text{且} \quad |f(x_1)| < \text{TOL2}.$$

注 B.2 算法中需要计算 $f(x)$ 和 $f'(x)$ 的子程序或过程.

例 B.1 应用Newton法求方程

$$f(x) = x^3 + 2x^2 + 10x - 20 = 0$$

在 $[1,2]$ 内的一个根.

解 由于 $f(1)=-7, f(2)=16$, 因此 $f(1)\cdot f(2)<0$. 又因 $f(x)$ 在 $[1,2]$ 上连续,

$$f'(x) = 3x^2 + 4x + 10 \neq 0, \quad x \in [1,2],$$

所以方程 $f(x)=0$ 在区间 $(1,2)$ 内有唯一根.

据 Newton 迭代公式

$$x_k = x_{k-1} - \frac{x_{k-1}^2 + 2x_{k-1}^2 + 10x_{k-1} - 20}{3x_{k-1}^2 + 4x_{k-1} + 10}, \qquad k = 1, 2, \cdots,$$

取初始值 $x_0 = 1$, 得

$$x_1 = x_0 - \frac{x_0^2 + 2x_0^2 + 10x_0 - 20}{3x_0^2 + 4x_0 + 10} = 1.411764706$$

$$x_2 = x_1 - \frac{x_1^3 + 2x_1^2 + 10x_1 - 20}{3x_1^2 + 4x_1 + 10} = 1.369336471$$

$$x_3 = 1.368808189,$$

$$x_4 = 1.368808108,$$

School of Math, NJU

此时 $|x_4 - x_3| = 8.1 \times 10^{-8}$, 可取近似根为 1.368808108. □

关于 Newton 法有下面的局部收敛性定理.

定理 A.1. *假设函数有 $n(>2)$ 阶连续导数, p 是方程 $f(x)=0$ 的单根, 则当 x_0 充分接近于 p 时, Newton法收敛, 且至少为二阶收敛.*

注 B.3 Newton 法的收敛性依赖于初值 x_0 的选取.

School of Math, NJU

参 考 文 献

[1] 林成森. 数值计算方法 (第二版 上、下册). 北京 科学出版社, 2005.

[2] 周铁, 徐树方, 张平文, 李铁军. 计算方法. 北京: 清华大学出版社, 2006.

[3] 李庆扬, 王能超, 易大义. 数值分析 (第五版). 北京: 清华大学出版社, 2008.

[4] 李荣华, 刘播. 微分方程数值解法 (第四版). 北京: 高等教育出版社, 2009.

[5] 黄云清, 舒适, 陈艳萍, 杨银, 魏华祎. 数值计算方法(第二版). 北京: 科学出版社, 2022.

[6] (美)金凯德 (Kincaid, D.), (美)切尼 (Cheney, W.). 数值分析 (影印版, 英文第三版). 北京: 机械工业出版社, 2003.

[7] 车刚明等编著. 数值分析典型题解析及自测试题 (第三版). 西安: 西北工业大学出版社, 2005.

[8] 现代应用数学手册编委会. 现代应用数学手册. 计算与数值分析卷. 北京: 清华大学出版社, 2005.

[9] 同济大学计算数学教研室编著. 现代数值计算. 北京: 人民邮电出版社, 2009.

[10] (美)索尔(Sauer, T.)著; 吴兆金, 王国英, 范红军译等. 数值分析. 北京: 人民邮电出版社, 2010.

[11] (美)马修斯(Mathews, J.H.)等著; 周璐等译. 数值方法(Matlab 版)(原书第四版). 北京: 电子工业出版社, 2005.

[12] 夸特罗尼(Alfio, Q)等著. 数值数学=Numerical Mathematics (影印版). 北京: 科学出版社, 2006.

[13] (美)伯登(Burden,R.L.)等. 数值分析 (第七版影印版). 北京: 高等教育出版社, 2001.

[14] Isaacson, E., Keller, H. Analysis of Numerical Methods, London – New York: John Wiley and Sons, 1967.

[15] Cheney,E. W.著; 王彭年等译. 逼近论导论 (Introduction to Approximation Theory). 北京: 高等教育出版社, 2000.

School of Math. NJU

[16] Hall, C.A., Meyer, W. Optimal error bounds for cubic spline interpolation. Journal of Approximation Theory. 16(2):105–122, 1976.

[17] Stoer, J., Bulirsch R. Introduction to Numerical Analysis, 3rd Edition, Springer, 2002.

[18] Mills,T.M, Smith S.J. A note on the Newton-Cotes integration formula Journal of Approximation Theory. 66(1):98-105, 1991.

[19] Hornik, K., Stinchcombe, M, White, H. Multilayer feedforward networks are universal approximators. Nerual Networks, 2(5): 359–366, 1989.

[20] Yarotsky, D. Error bounds for approximations with deep Relu networks. Neural Networks 94: 103 – 114, 2017.

[21] Maiorov,V.E. On Best Approximation by Ridge Functions. Journal of Approximation Theory, 99: 68-94, 1999.

[22] Brenner, S. C., Scott, L. R. The Mathematical Theory of Finite Element Methods (Third Edition). New York: Springer Science+Business Media, LLC, 2008.

School of Math, NJU